Stock Price Movement Prediction Using Hybrid LSTM- Transformer Models

I. Overview

This project aims to forecast the future trend in stock price movement through a deep learning model. The main issue in this work is to determine whether the stock will swing up or down within the upcoming trading period, using the historical market data. For this purpose, the model has been trained and tested, and the most optimal saved model has been chosen for the final deployment.

The primary aim was to apply the trained model to new unseen data and analyse its performance in another market period. Also, this step was also aimed at transforming the trained model into a simple and functional web application so that users can easily interact with it. Moreover, the entire testing, deployment and implementation process was recorded in a systematic manner.

The final application was deployed using Streamlit and run on Hugging Space hosting to enable access to it online. Thus, the project shifted from a notebook-based machine learning workflow to an operational web application, capable of making predictions in an interactive way.

II. Technical Details

A. Final Model Chosen for the Deployed Application

The deployed application uses the Transformer model because it gave the best overall result during the earlier model comparison stage [1]. Among the trained models, this model was selected as the final saved model because it provided a better balance in prediction performance. Additionally, using one selected model made the deployment process more stable and easier to manage.

B. Deployment Artefacts Used in the Application

Before the development of the web application, the main deployment files needed to be stored appropriately. The deployed application failed to work with just the model weights trained since it required the same scaling and configuration information with which it had been trained.

```
Files inside saved_models folder:
- transformer_best.pth
- model_config.json
- scaler.pkl
```

Figure 1: Saved Deployment Files

The files that have been saved are transformer_best.pth, model_config.json, and scaler.pkl. The weights of the final model are stored in the file transformer_best.pth. Similarly, scaler.pkl is needed to store the fitted scaler, which is required since all new market data is required to be transformed similarly to the training data. Additionally, the significant settings, including the name of the model, the length of the sequence, feature columns, input dimensions, and threshold value, are saved in model_config.json.

C. Preprocessing, Scaling, Model Loading, and Inference

The implemented system follows the same flow as in the development of the model. To begin with, the input market data is cleaned and organised in a proper format. Subsequently, feature scaling is done to ensure that the values of the inputs take the format of the trained model [2]. After that, the saved configuration is loaded, and the Transformer model is loaded using the saved weights. Lastly, the pre-planned input sequence is presented to the model, and its output is transformed into the prediction presented to the user.

D. Deployment Structure and Execution Setup

The project files were organised in a manageable and understandable system where the application would be properly executed over the internet. The repository contains the src folder that will hold the main application code and utility functions, and the saved models folder will hold the trained model and supporting materials. Also, README.md, requirements.txt and Dockerfile may assist with running the Streamlit app in Hugging Space.

III. Code Documentation

A. Main Application File

The deployed web application is mainly controlled by streamlit_app.py, and this file manages the complete user-side interaction of the system. It establishes the page structure, shows the name of the application, and the input box where the user types the ticker symbol and the lookback period to be used. Also, this file loads the saved model artefacts and only after clicking on the prediction button initiates the prediction process. This way, the application maintains a user flow to be easy and organised.

```python
def main():
    st.title("📈 Stock Direction Predictor")
    st.markdown("An interactive web app for forecasting stock movement direction using a trained deep learning model.")

    with st.sidebar:
        st.header("User Input")
        with st.form("prediction_form"):
            ticker = st.text_input("Enter Ticker Symbol", value="SPY").strip().upper()
            period = st.selectbox(
                "Select Lookback Period",
                options=["6mo", "1y", "2y", "5y"],
                index=1
            )
            run_button = st.form_submit_button("Run Prediction")

    model, scaler, config, device = load_artifacts()

    st.subheader("Model Information")
    col_a, col_b, col_c = st.columns(3)
    col_a.metric("Selected Model", config["best_model_name"])
    col_b.metric("Sequence Length", config["sequence_length"])
    col_c.metric("Decision Threshold", f'{config["best_threshold"]:.2f}')

    if run_button:
        try:
            with st.spinner("Downloading market data and generating prediction..."):
                raw_df = download_ticker_data(ticker, period)

                if raw_df.empty:
                    st.error("No data found for the selected ticker.")
                    return

                result = predict_latest_from_dataframe(
                    raw_df=raw_df,
                    model=model,
                    scaler=scaler,
                    config=config,
                    device=device
                )
```

Figure 2: Streamlit_app.py Code

The code section shows that streamlit-app.py receives user input, responsible for receiving user input through the sidebar

form and subsequently directing the user input into the prediction pipeline. It also demonstrates that the saved model, scaler, configuration and the device are loaded by the load_artefacts (), and it leads to the deployment file being directly related to the saved training outputs. The information on models is further reflected in the same file (selected model, sequence length, decision threshold) and assists the user in understanding the current prediction configuration [3]. It implies that not only is streamlit_app.py a display file, but it also serves as the central controller that connects the interface with the fundamental logic.

B. *Backend Utility File*

The second important file is model_utils.py, and this file will provide the processing necessary for prediction at the backend. It consists of helper functions to load the saved artefacts, prepare the input data and produce the final prediction output [4]. It also ensures that the similar reasoning that was applied in training is applied in deployment, which is required to have consistency between the notebook results and the web application results.

The load_pipeline_artefacts function loads model_config.json, scaler.pkl and the saved model weight file, and builds the final model in evaluation mode.

```python
def load_pipeline_artifacts(model_dir: str = "saved_models", device=None):
    if device is None:
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    config = load_config(f"{model_dir}/model_config.json")
    scaler = joblib.load(f"{model_dir}/scaler.pkl")

    model = build_model_from_config(config)
    model_name = config["best_model_name"].lower()
    model_path = f"{model_dir}/{model_name}_best.pth"

    state_dict = torch.load(model_path, map_location=device)
    model.load_state_dict(state_dict)
    model.to(device)
    model.eval()

    return model, scaler, config, device
```

Figure 3: Model Loading

In addition, predict_latest_from_dataframe() uses preprocessing, scaling, sequence preparation, probability calculation and final labelling.

```python
def predict_latest_from_dataframe(raw_df: pd.DataFrame, model, scaler, config: dict, device) -> dict:
    feature_cols = config["feature_cols"]
    sequence_length = config["sequence_length"]
    best_threshold = config["best_threshold"]

    processed_df = prepare_market_data(raw_df)

    scaled = scaler.transform(processed_df[feature_cols])
    scaled_df = pd.DataFrame(scaled, columns=feature_cols, index=processed_df.index)

    latest_seq = make_latest_sequence(scaled_df, feature_cols, sequence_length)
    X = torch.tensor(latest_seq, dtype=torch.float32).unsqueeze(0).to(device)

    with torch.no_grad():
        logits = model(X)
        probs = torch.softmax(logits, dim=1)[0].cpu().numpy()

    prob_down = float(probs[0])
    prob_up = float(probs[1])
    pred_class = int(prob_up >= best_threshold)
    pred_label = "Up" if pred_class == 1 else "Down"

    latest_row = processed_df.iloc[-1]

    if prob_up >= 0.65:
        confidence = "Strong"
    elif prob_up >= best_threshold + 0.03:
        confidence = "Moderate"
    else:
        confidence = "Weak"

    return {
        "prediction_date": latest_row["Date"],
        "latest_close": float(latest_row["Close"]),
        "predicted_class": pred_class,
        "predicted_label": pred_label,
        "prob_up": prob_up,
        "prob_down": prob_down,
        "threshold": float(best_threshold),
        "confidence": confidence,
        "processed_df": processed_df
```

Figure 4: Predictions

Therefore, model_utils.py acts as the technical core of the deployed system, while streamlit_app.py acts as the user-facing layer.

IV. Deployment Guide

A. *Preparing the Deployment Files*

The deployment process starts with the final trained model. To begin with, the Transformer model was saved from the notebook as a .pth file to reuse its trained weights in an environment other than training [5]. Once this was done, scaler.pkl and model_config.json were also saved. These support files were required because the deployed application needed to follow the same scaling rules, feature settings, and threshold values used to train the application. By doing so, the online system would generate predictions in the identical format as that of the notebook.

B. *Organising the Project Structure*

Once the necessary files had been ready, the next step was to set up the entire project in an appropriate deployment structure. The repository was required to hold the application code, dependency files and the saved model files in a distinct and understandable manner [6]. This was necessary since the online system had to establish a proper connection between the Streamlit interface, the backend utility functions and the saved artefacts.

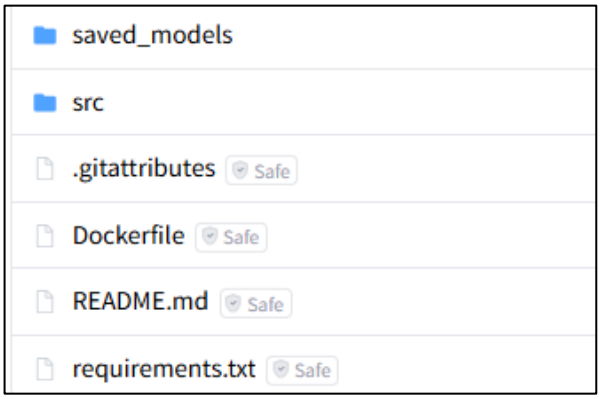

Figure 5: Hugging Face Repository Structure

The repository layout contains two main folders, saved models and src. The trained model and the deployment files are stored in the saved/models folder, and the code is in the src folder with application and utility code running at the time of execution. Moreover, .gitattributes, Dockerfile, README.md and requirements.txt are located on the top level of the repository. This demonstrates that the project was set up in an easy and practical manner to be deployed on Hugging Face Spaces.

C. *Running and Testing the Deployed Application*

After placing the files properly, the backend loading functions were used to link the Streamlit application and model files that were previously saved. This was followed by testing of the deployed application by providing sample input to the application and ensuring the correctness of the prediction process. The interface was also built in an easy manner such that the user could easily input the ticker symbol, select the period of lookback and read the model information displayed.

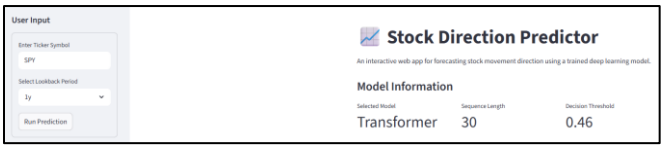

Figure 6: Deployed App Home Screen

The home screen shows a clean layout with a sidebar input section on the left and the main output area on the right. The ticker input box, lookback period option, and the prediction button are included in the sidebar, and all this combines to make the app interactive. Similarly, in the main region, the users can see the chosen model name, the length sequence followed, and, finally, the decision threshold to contribute to user understanding of the prediction arrangement before model execution.

V. Results and Analysis

A. Outcomes of Testing on New Data

Once the files were ready with the deployment, the Transformer model was tested on new, unknown data to determine whether it was still able to work similarly to the previous training scope [7]. This was a necessary step as the final system would need to be working with new market data and not just with the data that was used to initially develop the system. Also, this testing ensured that the comfortable model file, scaler file and configuration file were coordinating well in the final prediction pipeline.

```
New Data Performance:
Accuracy              0.554286
Balanced_Accuracy     0.454617
Macro_F1              0.454000
Weighted_F1           0.551611
ROC_AUC               0.459836
Precision_Class_0     0.224490
Recall_Class_0        0.215686
F1_Class_0            0.220000
Precision_Class_1     0.682540
Recall_Class_1        0.693548
F1_Class_1            0.688000
dtype: float64
```

Figure 7: New Data Performance Metrics

The performance values show that the model reached an accuracy of 55.43%, while the balanced accuracy was 45.46%. Similarly, Macro F1 was 0.454, and ROC AU was 0.460. These values demonstrate that the model produced average outcomes on unseen data, yet its generalisation was not very high. The model was also better at identifying an upward-moving direction as compared to a downward-moving direction since the precision, recall and F1 values of class 1 were significantly better than those of class 0.

B. Detailed Prediction Behaviour

The row-wise prediction table was also analysed in order to understand the testing outcome better. This was necessary since the summary metrics will not indicate at what point the model predicts correctly and where there are uncertainties.

	Date	Actual_Label	Predicted_Label	Predicted_Prob_Up	Forward_5D_Return	Volatility_20
0	2025-04-04	Up	Up	0.495559	0.056721	0.020239
1	2025-04-07	Up	Up	0.496265	0.068877	0.019672
2	2025-04-08	Up	Up	0.486612	0.082843	0.019806
3	2025-04-09	Down	Up	0.501237	-0.041851	0.031634
4	2025-04-10	Up	Down	0.457517	0.003489	0.033000
5	2025-04-11	Down	Up	0.475921	-0.037570	0.032905
6	2025-04-14	Down	Up	0.482037	-0.022017	0.032939
7	2025-04-15	Down	Up	0.490994	-0.004074	0.032873
8	2025-04-16	Up	Up	0.491161	0.040007	0.033051
9	2025-04-17	Up	Up	0.510996	0.046029	0.033067
10	2025-04-21	Up	Up	0.500620	0.071943	0.033381
11	2025-04-22	Up	Up	0.510339	0.051342	0.033710
12	2025-04-23	Up	Up	0.497819	0.035710	0.033961
13	2025-04-24	Up	Up	0.499102	0.021548	0.034307
14	2025-04-25	Up	Up	0.485820	0.029275	0.034359

Figure 8: New Data Prediction Table

The table shows that several upward cases were predicted correctly, such as 2025-04-04, 2025-04-07, and 2025-04-08. However, many downward cases were still predicted as upward, such as 2025-04-09, 2025-04-11, 2025-04-14, and 2025-04-15. Also, most probabilities are close to the range of thresholds of about 0.46-0.51.

C. Behaviour of the Deployed Application

The application deployed was also verified after viewing the output of the notebook with a sample ticker input. This was necessary since the final system must be able to operate well not just within the notebook, but also through the online interface. The implemented application provided a positive forecast, including probabilities and market data to use.

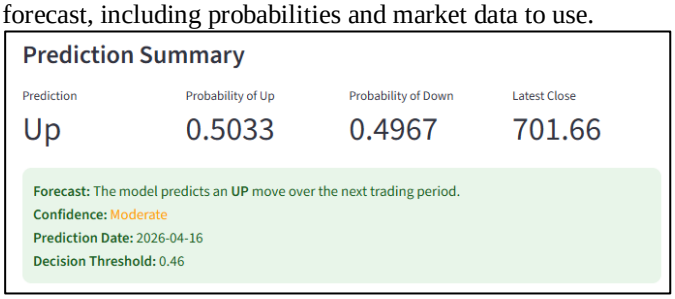

Figure 9: Deployed App Prediction Summary

An upward probability of 50.33% and a downward probability of 49.67% was predicted in the forecast of the outcomes. Such a small gap implies that the prediction of the model was just slightly above the threshold, and there was not very strong confidence.

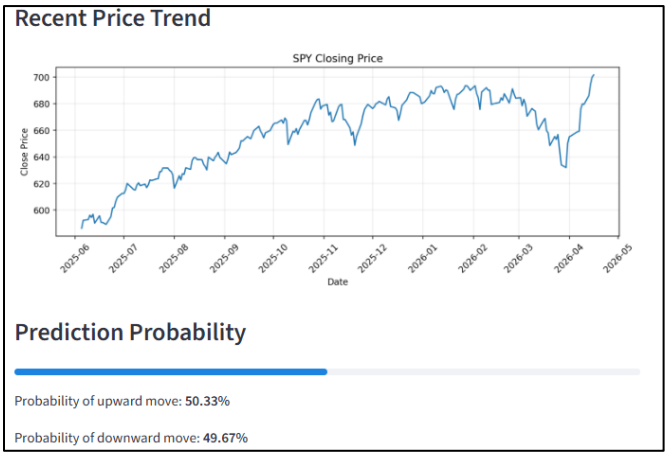

Figure 10: Recent Price Trend

The recent price trend shows that SPY moved upward overall during the selected period, even though there was a noticeable fall before the latest recovery. This recent upward movement helps explain why the model leaned toward an upward prediction.

Key Market Indicators

Metric	Value
Close	701.66
1-Day Return	0.0025
5-Day Return	0.032
RSI (14)	70.4452
Stochastic %K	98.4761
MACD Diff	6.2596
ATR / Close	0.013
Volatility (20)	0.0119

Figure 11: Key Market Indicators

This finding is also validated by the indicator values, since the RSI is approximately 70.45, the Stochastic %K is high, and the difference of MACD is positive. A combination of these values indicates a powerful short-term positive momentum.

Recent Market Data

Date	Close	1-Day Return	5-Day Return	RSI (14)	MACD Diff	Volatility (20)
2026-04-02 00:00:00	655.83	0.0009	0.0166	46.4085	0.3748	0.0118
2026-04-06 00:00:00	658.93	0.0047	0.0392	48.5259	1.1921	0.0116
2026-04-07 00:00:00	659.22	0.0004	0.0431	48.7299	1.7359	0.0114
2026-04-08 00:00:00	676.01	0.0255	0.0395	58.8918	3.1287	0.0128
2026-04-09 00:00:00	679.91	0.0058	0.0377	60.8337	4.1509	0.0129
2026-04-10 00:00:00	679.46	-0.0007	0.036	60.4787	4.6003	0.0124
2026-04-13 00:00:00	686.1	0.0098	0.0412	63.8326	5.1089	0.0124
2026-04-14 00:00:00	694.46	0.0122	0.0535	67.5647	5.7262	0.0125
2026-04-15 00:00:00	699.94	0.0079	0.0354	69.767	6.1823	0.0125
2026-04-16 00:00:00	701.66	0.0025	0.032	70.4452	6.2596	0.0119

Figure 12: Recent Market Data

According to the recent market information, the closing price and positive returns of the past 5 days in the previous rows are steadily growing. Thus, both the indicator values and the recent market movement support the application output.

D. Challenges Faced During Testing and Deployment

One important challenge was that the model failed to generalise well on the unseen data, particularly when there was downward movement and numerous predictions remained very close to the limit. Moreover, at the beginning of the deployment process, there were certain problems with file location, execution route, and the relationship between the app and the saved model files. These issues were resolved by choosing the most appropriate saved model, setting the saved threshold in the configuration file, organising the repository and placing proper folders, fixing the app execution path and retesting the system with every update.

VI. Conclusion

This project provided helpful guidance on applying a deep learning model, not only in developing the model but also in designing a working web application. One important insight from this work was that testing on unknown new data would be needed, as it provides a more realistic perspective of model behaviour. Also, the project demonstrated that the trained model is not the only factor that can successfully deploy the project, proper preprocessing, saved configuration files, and correct system setup all play a crucial role.

The overall effect of such a project is that it resulted in a complete prediction system that integrates testing, deployment and user interaction within a single platform. The application can produce predictions of stock movements and provide corresponding market information in an interactive and understandable format. This work can be enhanced in the future by developing models that achieve better generalisation, support a greater number of assets, and improve the interface to make the output of the prediction process even easier to interpret and use.

VII. References

[1] L. Torbarina, T. Ferkovic, L. Roguski, V. Mihelcic, B. Sarlija and Z. Kraljevic, "Challenges and Opportunities of Using Transformer-Based Multi-Task Learning in NLP Through ML Lifecycle: A Position Paper.," *Natural Language Processing Journal*, p. p.100076, 2024.

[2] M. Ahsan, M. Mahmud, P. Saha, K. Gupta and Z. Siddique, "Effect of Data Scaling Methods on Machine Learning Algorithms and Model Performance," *Technologies*, vol. 9, no. 3, p. 52, 2021.

[3] M. Ali, A. Khattak, Z. Ali, B. Hayat, M. Idrees, Z. Pervez, K. Rizwan, T.-E. Sung and K. Kim, "Estimation and Interpretation of Machine Learning Models with a Customised Surrogate Model," *Electronics*, vol. 10, no. 23, p. p.3045., 2021.

[4] J. Çarka, M. Esposito and D. Falessi, "On effort-aware metrics for defect prediction. Empirical Software Engineering," vol. 27, no. 6, 2022.

[5] K. Spoon, H. Tsai, A. Chen, M. Rasch, S. Ambrogio, C. Mackin, A. Fasoli, A. Friz, P. Narayanan, M. Stanisavljevic and G. Burr, "Toward Software-Equivalent Accuracy on Transformer-Based Deep Neural Networks With Analog Memory Devices," *Frontiers in Computational Neuroscience*, vol. 15, 2021.

[6] Y. Tao, Y. Qin and Y. Liu, "Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches," 2025.

[7] A. Ibias, M. Méndez, M. Núñez and F. Palomo-Lozano, "Using transformers to learn system models," *Applied Intelligence*, vol. 56, no. 3, p. 71, 2026.

DEPLOYED MODEL –
App link
https://huggingface.co/spaces/syedmajid254344567/Stock_Price_Movement_Flash_Forecasting

Colab python code link –
https://colab.research.google.com/drive/1kJv6b73nnpz_j6y8LGLBN6ok6K4jz23h?usp=sharing